

INTRODUCCIÓN A SOLIDITY: EL LENGUAJE DE LOS CONTRATOS

Para quienes ya programan en Java, Python o JavaScript y nunca han escrito un contrato. No es un laboratorio: es el documento que se consulta cuando algo no compila, no cuadra o no se entiende.

PARA QUIÉN	VERSIÓN	CÓMO SE USA	CIFRAS DE GAS
Quien ya programa y empieza en Solidity	Solidity 0.8.34 OpenZeppelin 5.6.1	De consulta, no de corrido	Medidas aquí, no estimadas

CÓMO LEER ESTA GUÍA

No se lee de corrido la primera vez. Lean la sección **01** antes de la Sesión 6 —es la que cambia la cabeza— y después vuelvan a lo que les haga falta. Las secciones **14** (errores del primer mes) y **16** (glosario) son las que más van a usar.

Todo el código de esta guía compila con 0.8.34 tal como está: se compiló antes de imprimirla. **Todas las cifras de gas se midieron** en la red local de este repositorio (Hardhat 3.16.0, Solidity 0.8.34, optimizador con 200 ejecuciones); no salen de ningún blog. La red local es determinista: si repiten la medición, obtienen los mismos números.

Aquí se enseña **el lenguaje**. Los laboratorios 06 y 07 enseñan a **usarlo** —Remix, despliegue, verificación, patrones de fondos, medición de gas— y no se repite aquí lo que está allá.

01 Qué es Solidity, y en qué NO se parece a lo que ya saben LEER PRIMERO

Solidity es un lenguaje compilado, de tipado estático, con clases (se llaman *contratos*), herencia, interfaces y bibliotecas. Si vienen de Java o de TypeScript, buena parte de la sintaxis les resultará familiar desde el primer minuto: `if`, `for`, `while`, `return`, llaves, punto y coma, `//` y `/* */`. Ese parecido es la trampa: lo que no se parece viene de tres hechos sin equivalente en ningún lenguaje que hayan usado antes.

LOS TRES HECHOS QUE LO CAMBIAN TODO

1 · El programa no corre en su computador: corre en una máquina compartida por todo el mundo. No hay `main()`, ni proceso, ni hilo. El contrato está dormido hasta que alguien le envía una transacción; se despierta, ejecuta una función, guarda lo que cambió y se vuelve a dormir. Entre una llamada y otra pueden pasar años, así que todo lo que quieran recordar tiene que estar **escrito en el estado**: no hay caché en RAM que sobreviva.

2 · Cada instrucción cuesta dinero. Quien llama paga, en ether, cada operación que se ejecuta. Un bucle sobre 10 000 elementos no es «lento», es **imposible**: no cabe en el límite de gas del bloque. Eso invierte reflejos —se guarda poco y se calcula afuera, no al revés—.

3 · El código desplegado es inmutable y público. No hay «subir un parche el viernes»: lo que se despliega queda ahí para siempre, y cualquiera puede leer el bytecode y **todo el estado**, incluidas las variables `private`. Se parece más a fabricar un chip que a desplegar un servicio web.

LO QUE YA SABEN	EN SOLIDITY	LA DIFERENCIA QUE MUERDE
Clase	<code>contract</code>	Una sola instancia, con dirección propia, desplegada una vez.
Campos de la clase	Variables de estado	Persisten para siempre; escribir una cuesta hasta 20 000 de gas.
<code>int</code> , <code>long</code>	<code>uint8 ... uint256</code> , <code>int8 ... int256</code>	El tamaño por defecto es de 256 bits , no de 32.
<code>double</code> , <code>float</code>	No existen	Solo enteros. Sección 13.
<code>HashMap</code> , <code>dict</code> , <code>Map</code>	<code>mapping</code>	No se recorre, no tiene tamaño, no se sabe qué claves tiene.
<code>null</code> , <code>None</code>	No existen	Todo tiene valor por defecto: cero, <code>false</code> , cadena vacía.
<code>try/catch</code> <code>local</code>	<code>revert</code>	No se atrapa dentro de la función: deshace toda la transacción.
<code>println</code> , <code>console.log</code>	No existe	Un contrato no tiene consola. Lo más parecido son los eventos (sección 09).
<code>private</code>	<code>private</code>	Limita quién puede llamar , no quién puede ver . Sección 06.
<code>Math.random()</code> , leer un archivo, llamar una API	No existen	Una máquina determinista y aislada no puede sortear ni salir a internet. Eso lo hacen los oráculos (Sesión 14).
Hora del sistema	<code>block.timestamp</code>	Es la hora que declaró quien produjo el bloque, y se puede mover.

La pregunta que hay que hacerse siempre, antes de escribir una línea: ¿este dato tiene que estar en la cadena? Lo que va a la cadena es público para siempre, cuesta caro de escribir y no se borra. Muchas veces la respuesta honesta es que no. Un contrato bien diseñado guarda lo mínimo para que las reglas se cumplan sin confiar en nadie, y deja el resto afuera.

02 Anatomía de un archivo `.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

/// @title Saludo mínimo
/// @author Curso Blockchain y Web 3.0 · USB Medellín
/// @notice Guarda un saludo y dice quién lo puso.
/// @dev Material de clase, sin auditar.
contract Saludo is Ownable {
    string public texto;

    constructor(string memory inicial) Ownable(msg.sender) { texto = inicial; }

    /// @notice Cambia el saludo.
    /// @param nuevo el texto que queda guardado
    function cambiar(string calldata nuevo) external onlyOwner { texto = nuevo; }
}
```

LÍNEA	QUÉ ES Y POR QUÉ ESTÁ AHÍ
SPDX	Licencia del código fuente ; la única línea que puede ir antes del <code>pragma</code> . Si falta, el compilador avisa (advertencia, no error) y el explorador muestra «licencia no especificada» al verificar. Valores usuales: <code>MIT</code> , <code>GPL-3.0</code> , <code>Apache-2.0</code> , <code>UNLICENSED</code> (este último significa «nadie puede reutilizar esto»).
pragma	Qué versiones del compilador aceptan este archivo. Es un filtro que se comprueba al compilar , no una importación.
import	Trae símbolos de otro archivo. Siempre con llaves : sin ellas se vuelca todo al espacio de nombres actual —incluido lo que ese archivo importó— y después nadie sabe de dónde salió cada símbolo. Las rutas con <code>@</code> las resuelve el gestor de paquetes desde <code>node_modules</code> ; Remix las descarga solo.
NatSpec	Los comentarios <code>///</code> y <code>/** */</code> con etiquetas <code>@</code> . No son adorno: el compilador los guarda en el ABI y en los metadatos, y las billeteras se los muestran al usuario al firmar. Cuando MetaMask dice «¿Confirma cambiar el saludo?» en lenguaje humano, está leyendo esto. Trampa comprobada : si una línea del bloque <i>empieza</i> con una arroba que no es una etiqueta conocida, el compilador falla con <code>DocstringParsingError: Documentation tag @... not valid</code> . Una arroba a mitad de línea (un correo, por ejemplo) no molesta; una al principio, sí.
contract	La unidad desplegable. Un archivo puede tener varios; se despliegan por separado.

POR QUÉ SE ESCRIBE `pragma solidity 0.8.34;` y no `^0.8.0`

El circunflejo significa «0.8.0 o superior, pero menor que 0.9.0». Es lo que verán en casi todos los tutoriales, es cómodo, y esconde un problema serio: **el mismo código compilado con dos versiones distintas produce bytecode distinto**. De ahí salen tres dolores: prueban con 0.8.19 y despliegan con 0.8.34, así que lo que auditaron no es lo que quedó en la cadena; la verificación en el explorador **falla** si no aciertan la versión exacta, y después no hay forma de adivinarla; y dos personas del mismo equipo obtienen bytecode distinto «sin haber cambiado nada». Fijar la versión es gratis y elimina los tres. En bibliotecas que otros van a importar sí tiene sentido el rango —por eso OpenZeppelin lo usa—; en **su** contrato desplegable, no.

Y hay un segundo efecto que casi nadie menciona: la versión del compilador también decide PARA QUÉ EVM se genera el código. Cada compilador trae un *objetivo de EVM* (*EVM target*) por omisión, que es el conjunto de opcodes que da por disponibles. Este curso vio el cambio en vivo: al pasar de `0.8.28` a `0.8.34` el objetivo por omisión pasó de `cancun` a `osaka`, y Hardhat lo dice en una línea al compilar —`Compiled N Solidity files with solc 0.8.34 (evm target: osaka)`—. Vale la pena leer esa línea: si el objetivo es más nuevo que la red donde van a desplegar, el contrato puede usar un opcode que allá no existe y fallar al desplegarse o al ejecutarse. Es otra razón para fijar la versión y anotarla: «mi contrato» no es solo el código fuente, es el código fuente **más** el compilador **más** el objetivo de EVM.

03 Los tipos

LA TABLA QUE MÁS SE CONSULTA

TIPO	POR DEFECTO	QUÉ HAY QUE SABER
<code>uint8 ... uint256</code>	<code>0</code>	Sin signo, en pasos de 8 bits. <code>uint</code> = <code>uint256</code> , que llega hasta $2^{256}-1$.
<code>int8 ... int256</code>	<code>0</code>	Con signo. <code>int</code> = <code>int256</code> . Se usan poco: el dinero de la EVM no es negativo.
<code>bool</code>	<code>false</code>	Ocupa 1 byte en storage, no 1 bit.
<code>address</code>	<code>address(0)</code>	20 bytes. Puede ser una persona o un contrato, y desde dentro no hay forma fiable de distinguirlas . <code>address(0)</code> es «el vacío» y también el destino de las quemas de tokens.
<code>address payable</code>	<code>payable(address(0))</code>	Un <code>address</code> que admite <code>.transfer()</code> y <code>.send()</code> . Se obtiene con <code>payable(a)</code> .
<code>bytes32</code>	32 ceros	Tamaño fijo, cabe justo en una ranura : lo más barato para un hash o un identificador. Se compara con <code>==</code> .
<code>bytes</code>	<code>""</code>	Tamaño variable, con <code>.length</code> . Para datos binarios de largo desconocido.
<code>string</code>	<code>""</code>	Tamaño variable, UTF-8. No tiene <code>.length</code> , no se indexa y no se compara con <code>==</code> .
<code>enum</code>	el primer valor	Azúcar sobre un entero pequeño: <code>enum Estado { Libre, Reservada } → 0, 1</code> . Las pruebas lo ven como número.

<code>struct</code>	todo en cero	Agrupar campos. El orden de declaración decide cuántas ranuras ocupa (sección 05).
<code>T[3]</code> (fijo)	3 valores en cero	Longitud conocida al compilar. No tiene <code>.push</code> .
<code>T[]</code> (dinámico)	<code>[]</code>	<code>.length</code> , <code>.push(x)</code> , <code>.pop()</code> . Recorrerlo en cadena es caro y se vuelve imposible si crece sin límite.
<code>mapping(K => V)</code>	ver abajo	Tabla clave → valor. Solo existe en <code>storage</code> : no puede ser parámetro ni variable local.

LAS TRES RAREZAS DEL `mapping`

1 · No se recorre. No existe «dame todas las claves», no tiene `.length`, no se itera. La razón es física: un `mapping` no guarda las claves en ninguna parte; la posición de cada valor se calcula con un hash. Si necesitan la lista, mantengan un arreglo aparte **a mano** (`if (puntos[quien] == 0) _vistos.push(quien);`) y paguen por ello.

2 · Toda clave existe desde siempre, con valor cero. Preguntar por una dirección que nunca apareció devuelve `0`, no un error ni `null`. Por eso «¿existe este registro?» **no** se puede responder con «¿el valor es distinto de cero?»: hace falta un campo `bool existe` explícito. Y `delete m[k]` no «quita la clave»: la pone en cero, que es lo mismo que nunca haber estado.

3 · Las claves se pueden nombrar desde 0.8.18: `mapping(address duenno => uint256 saldo) public saldos;` Es solo documentación —genera lo mismo— pero se lee mucho mejor y los nombres llegan al ABI.

3.1 · `bytes32` frente a `bytes` frente a `string`

`bytes32` es de tamaño fijo, ocupa **una ranura exacta** y se compara con `==`: es lo más barato para hashes, identificadores y claves. `bytes` es de tamaño variable (una ranura de cabecera más los datos) y sirve para binario de largo desconocido: una firma, un `calldata` guardado. `string` cuesta lo mismo que `bytes` y de hecho es un `bytes` con la promesa de ser UTF-8, promesa que el compilador no verifica.

Solidity casi no sabe manipular texto: no hay `substring`, ni `toUpperCase`, ni `split`. Lo que sí hay:

```
// Los strings NO se comparan con ==. Se comparan sus hashes.
return keccak256(bytes(x)) == keccak256(bytes(y));

return bytes(x).length == 0; // un string no tiene .length
```

Y el detalle que muerde: `bytes(x).length` cuenta **bytes**, no caracteres. En UTF-8 «ñ» ocupa dos bytes y un emoji cuatro. Un límite de 100 no es un límite de 100 letras.

3.2 · Conversiones y truncamiento

```
function conversiones(uint256 grande) external pure returns (uint8 a, uint16 b) {
    a = uint8(grande); // con grande = 300 devuelve 44, SIN avisar
    b = uint16(a);     // ensanchar nunca pierde nada
}
```

Estrechar trunca en silencio. 300 en binario es `0000_0001_0010_1100`; `uint8` se queda con el último byte, `0010_1100` = 44. No hay error, ni advertencia, ni reversión: el programa sigue con un número falso. Es la fuente de errores más silenciosa del lenguaje, y aparece sobre todo al convertir `block.timestamp` (que es `uint256`) a `uint64` o `uint32`. Hay media protección: escribir `uint8(300)` con el *literal no compila* («Explicit type conversion not allowed from "int_const 300" to "uint8"»), pero si el 300 viene dentro de una **variable** el compilador no puede saberlo y la conversión pasa, truncando. La protección cubre los literales, no los datos.

04 Dónde vive un dato: `storage`, `memory`, `calldata`

CON CIFRAS MEDIDAS

En Java o Python no hay que decir dónde vive un objeto. En Solidity, para todo lo que tenga tamaño variable (arreglos, `struct`, `string`, `bytes`) hay que declararlo, y esa palabra decide el comportamiento y el precio.

ZONA	CUÁNTO DURA	SE PUEDE	PRECIO
<code>storage</code>	Para siempre, en la cadena	leer y escribir	Carísimo: 20 000 por estrenar una ranura, 2 900 por cambiar una que ya tenía valor, y 2 100 más la primera vez que se toca esa ranura en la transacción («en frío»). Leer: 2 100 en frío, 100 después.
<code>memory</code>	Lo que dura la llamada	leer y escribir	Barato, no gratis: 3 de gas por palabra de 32 bytes, y sube al cuadrado a medida que crece.
<code>calldata</code>	Lo que dura la llamada	solo leer	Lo más barato: no se copia nada, los datos ya vienen en la transacción.

EL ERROR QUE COMPILA, NO REVIERTE Y NO HACE NADA

```
function usar(uint256 id) external {
    Ficha storage f = _fichas[id];
    f.usada = true;           // QUEDA escrito
}

function usarMal(uint256 id) external view returns (bool) {
    Ficha memory f = _fichas[id];
    f.usada = true;          // compila, no revierte y NO cambia nada
    return f.usada;          // devuelve true... y el storage sigue igual
}
```

Es el error número uno de quien empieza, y el más difícil de ver porque **no da ningún síntoma**: la función no falla e incluso devuelve lo que uno espera. Solo lo detecta la prueba que vuelve a leer el estado. Regla: **si lo van a modificar, va en `storage`**.

La misma función —suma un arreglo y guarda el resultado— con el parámetro de las dos formas, medida:

TAMAÑO DEL ARREGLO	<code>calldata</code>	<code>memory</code>	LO QUE CUESTA COPIAR
8 elementos	29 426	30 198	+772 (+2,6 %)
32 elementos	37 034	39 848	+2 814 (+7,6 %)

La diferencia **crece con el tamaño, y más rápido que el tamaño**, porque la memoria se cobra al cuadrado. Con 64 elementos, escribir `memory` en lugar de `calldata` sale 11,7 % más caro sin ganar nada. (Los totales incluyen los 21 000 fijos de cualquier transacción: lo que importa es la diferencia entre filas. Sin optimizador, la de 64 sube a 14 564.)

La regla: en funciones `external`, los parámetros de tamaño variable van en `calldata`, salvo que haya que modificarlos dentro (`calldata` es de solo lectura). En funciones `public` no se puede —como también se llaman desde dentro, el parámetro debe ser `memory`—, y esa es una buena razón para preferir `external`.

05 Estado, locales, y las ranuras de 32 bytes

Variable de estado: se declara dentro del `contract` y fuera de toda función; vive en `storage`, persiste entre transacciones y se paga cada escritura. **Variable local:** se declara dentro de una función; vive en la pila o en memoria, desaparece al terminar y es prácticamente gratis.

	CUÁNDO SE FIJA EL VALOR	¿OCUPA RANURA?	COSTO DE LEERLA
normal	En cualquier momento	Sí	2 100 en frío, 100 en caliente
constant	Al escribir el código	No	Casi cero: el valor se incrusta en el bytecode
immutable	En el constructor, una vez	No	Casi cero: el valor se incrusta al desplegar

Una `immutable` **solo** se escribe en el constructor y **no se puede leer dentro de él**: en ese momento el valor todavía no está en el bytecode. Una `constant` no puede depender de nada de tiempo de ejecución (ni `block.timestamp`, ni `msg.sender`).

5.1 • El empaquetado, con la prueba delante

El storage es una lista de ranuras de 32 bytes numeradas desde 0. El compilador coloca las variables de estado **en el orden en que se declaran** y mete varias en la misma ranura si caben; cuando no caben, empieza una nueva. Un `uint256` ocupa la ranura entera.

```
// CABEN EN UNA SOLA RANURA: 20 + 8 + 1 = 29 bytes de 32
address public autor;      // 20 bytes -> ranura 0
uint64  public fecha;      // 8 bytes -> ranura 0
bool    public activo;     // 1 byte  -> ranura 0

// NO EMPAQUETA: cada uno estrena ranura
uint256 public total;      // ranura 1 (ocupa las 32)
uint128 public a;          // ranura 2
uint256 public otro;       // ranura 3 <- se cuela y rompe el grupo
uint128 public b;          // ranura 4 (a y b habrían compartido ranura)
```

No es teoría. Se desplegó ese contrato, se le dieron valores reconocibles y se leyeron las ranuras con `eth_getStorageAt`:

```

ranura 0: 0x00000001bbbbbbbbbbbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
ranura 1: 0x000000000000000000000000000000000000000000000000cc
ranura 2: 0x00000000000000000000000000000000000000000000000dd
ranura 3: 0x00000000000000000000000000000000000000000000000ee
ranura 4: 0x00000000000000000000000000000000000000000000000ff

```

En la ranura 0 están los tres primeros, pegados de derecha a izquierda: la dirección (`aaaa...` , 20 bytes), la fecha (`bbbb...` , 8 bytes) y el `bool` (`01`). Los otros cuatro ocupan una ranura cada uno: `a` y `b` , que juntos ocupaban 32 bytes exactos, quedaron separados **solo porque se declaró un `uint256` entre ellos**.

Lo que cuesta esa distracción, medido sobre dos contratos con los mismos tres datos (`address` + `uint64` + `bool`) escribiéndolos en una sola llamada:

CONTRATO	RANURAS ESCRITAS	GAS	DIFERENCIA
Declaraciones seguidas	1	44 352	—
Con un <code>uint256</code> en medio	2	66 463	+22 111 (+49,9 %)

Esos 22 111 son casi exactamente el precio de estrenar una ranura: 20 000 de la escritura más 2 100 del primer acceso. Reordenar tres líneas de declaración ahorró la mitad del costo de la función, en cada llamada y para siempre.

QUÉ EMPAQUETA Y QUÉ NO

- **Empaquetan:** variables *consecutivas* cuyos tamaños sumen 32 bytes o menos. `address` (20) + `uint64` (8) + `uint32` (4) = 32 justos; cuatro `uint64` seguidos; 32 `bool` seguidos (un `bool` ocupa 1 byte, no 1 bit).
- **No empaquetan nunca:** `uint256` , `int256` y `bytes32` , que ocupan la ranura entera. Los arreglos dinámicos, los `mapping` , los `string` y los `bytes` usan su propia ranura de cabecera y guardan el contenido en otra parte.
- **Las locales no se empaquetan** y su orden no importa: viven en la pila, que trabaja en palabras de 32 bytes. Usar `uint8` en una local no ahorra nada y a veces cuesta *más*, porque el compilador añade instrucciones para recortar el valor.
- El empaquetado solo paga si las variables **se escriben juntas** en la misma transacción.

UN CONSEJO MUY REPETIDO QUE HOY YA NO PAGA · MEDIDO

«Copien la variable de estado a una local antes de usarla varias veces». Se midió una función que usa `base` cinco veces, frente a la misma con `uint256 b = base;` al principio. **Con el optimizador activado, como en este curso: 28 755 frente a 28 782, o sea nada** —esos 27 de gas son ruido—, porque el optimizador ya hace la copia solo. **Sin optimizador: 29 697 frente a 29 335, 362 de ahorro**, que son los cuatro `SLOAD` en caliente de 100 que se evitan. La copia local sigue siendo más legible y protege de leer un valor que cambió a mitad de función; lo que no se sostiene es el argumento del gas. **Hay que medirlo, no repetirlo:** muchos consejos de optimización que circulan son de antes del optimizador actual.

06 Funciones

```
function nombre(tipo parámetro) visibilidad mutabilidad modificadores returns (tipo) { ... }
```

VISIBILIDAD	QUIÉN PUEDE LLAMARLA	CUÁNDO SE USA
external	Solo desde fuera (o desde dentro con <code>this.f()</code>).	La opción por defecto para la interfaz pública. Permite <code>calldata</code> .
public	Desde fuera y desde dentro.	Cuando de verdad hace falta llamarla por dentro también.
internal	Este contrato y los que hereden de él.	Ayudantes compartidos. Es el valor por defecto de las variables de estado.
private	Solo este contrato. Ni los herederos.	Detalles internos que no deben tocarse al heredar.

Una variable de estado `public` no es «modificable por cualquiera»: el compilador le genera **gratis** una función de lectura con su mismo nombre. Escribirla sigue siendo asunto exclusivo del código del contrato.

private NO ES SECRETO · LA ADVERTENCIA MÁS IMPORTANTE DE ESTA GUÍA

`private` e `internal` limitan **quién puede llamar o referenciar** un miembro desde otro código Solidity. No esconden nada: todo el storage de un contrato es público, y cualquier nodo responde a `eth_getStorageAt(dirección, ranura)` devolviendo los 32 bytes que haya ahí, sin permiso de nadie. Se comprobó desplegando este contrato con `clave = 1234` y leyendo la ranura 0 desde fuera, sin ABI:

```
contract Secreto {
    uint256 private _clave;          // "privada": nadie la puede LLAMAR
    constructor(uint256 clave) { _clave = clave; }
}

ranura 0 → 0x000000000000000000000000000000000000000000000000000000000000004d2 = 1234
```

Nunca se guarda un secreto en una cadena pública. Ni una contraseña, ni una clave de API, ni la respuesta de un sorteo antes de revelarla, ni un dato personal «porque está en una variable privada». Si algo tiene que ser secreto y estar en cadena, se guarda su **hash** y el valor se revela después (patrón *commit-reveal*).

MUTABILIDAD	SE COMPROMETE A	CONSECUENCIA
(ninguna)	Puede escribir el estado.	Siempre exige una transacción y siempre cuesta gas.
view	Lee el estado, no lo escribe.	Llamada desde fuera: la responde el nodo, sin transacción y sin costo .
pure	Ni lee ni escribe el estado.	Igual que <code>view</code> , y además se razona sobre ella como una función matemática.

«Gratis» describe a quien pregunta, no a la función. Una `view` es gratis cuando la llama una interfaz desde fuera, porque el nodo la ejecuta en su copia sin escribir nada. Pero si **otro contrato** la llama dentro de una transacción, el cómputo se ejecuta de verdad y se paga. Las mismas líneas, dos precios, según quién pregunte.

6.1 · Retornos múltiples, sobrecarga, y llamarse a sí mismo

```
/// Dos valores de retorno, con nombre: no hace falta escribir `return`.
function partir(uint256 x) external pure returns (uint256 mitad, uint256 resto) {
    mitad = x / 2;
    resto = x % 2;
}

/// Sobrecarga: mismo nombre, firmas distintas.
function sumar(uint256 x, uint256 y) public pure returns (uint256) { return x + y; }
function sumar(uint256 x, uint256 y, uint256 z) public pure returns (uint256) { return x + y + z; }

uint256 s = sumar(x, 1);           // llamada INTERNA: un salto, casi gratis
uint256 t = this.desdeFuera();     // llamada EXTERNA a sí mismo: ~700 gas más
```

Los retornos múltiples se reciben con paréntesis — `(uint256 m, uint256 r) = partir(7);` — y se descarta lo que no interese dejando el hueco: `(, uint256 r) = partir(7);`. La sobrecarga funciona dentro de Solidity, pero desde fuera **las dos funciones tienen selectores distintos** y las bibliotecas cliente obligan a escribir la firma completa `(contrato["sumar(uint256,uint256)"] (...))`: usarla poco ahorra dolores.

La última línea cuesta: **26 650 frente a 27 349 de gas, 699 de diferencia**, medido. Una llamada interna es un salto dentro del mismo bytecode; escribir `this.f()` la convierte en un mensaje de verdad, con codificación ABI, opcode `CALL` y decodificación del retorno. Solo se justifica para llamar una función `external` desde dentro, o cuando se quiere a propósito que `msg.sender` pase a ser el propio contrato —lo cual, hecho sin darse cuenta, rompe el control de acceso—.

07 Las variables globales

EL CONTEXTO DE LA LLAMADA

No se declaran: están siempre disponibles. Son la única ventana que tiene el contrato al mundo.

VARIABLE	TIPO	QUÉ ES · Y LA TRAMPA
<code>msg.sender</code>	address	Quién hizo esta llamada. Si una persona llama a A y A llama a B, dentro de B <code>msg.sender</code> es A , no la persona. Es la base de todo control de acceso.
<code>msg.value</code>	uint256	Cuántos wei vienen en esta llamada. Solo tiene sentido en funciones <code>payable</code> .
<code>msg.data</code>	bytes calldata	Los datos completos: 4 bytes de selector + argumentos. Es lo que lee un <code>fallback</code> . <code>msg.sig</code> es solo el selector.
<code>block.timestamp</code>	uint256	Segundos Unix que declaró quien produjo el bloque. No es un reloj exacto: se puede mover unos segundos. Sirve para plazos de minutos u horas; no sirve para sortear nada ni para decidir quién llegó primero por medio segundo.

<code>block.number</code>	uint256	Altura del bloque. Avanza de a uno, pero el tiempo entre bloques no es constante entre redes: contar bloques no es contar tiempo.
<code>block.chainid</code>	uint256	Identificador de red (1 = Ethereum, 11155111 = Sepolia). Hace que una firma de una red no valga en otra (Sesión 13).
<code>tx.origin</code>	address	La cuenta de persona que inició toda la cadena de llamadas. Nunca para autorizar.
<code>address(this)</code>	address	La dirección del propio contrato. <code>address(this).balance</code> da su saldo en wei.
<code>gasleft()</code>	uint256	Cuánto gas queda en esta ejecución. Para medir, y para decidir si alcanza para una llamada externa.

POR QUÉ `tx.origin` NO SIRVE PARA AUTORIZAR

Suena razonable: «solo el dueño puede llamar, así que compruebo `tx.origin == dueño`». El ataque tiene tres pasos: (1) el atacante despliega un contrato cuyo `receive()` llama a `retirarTodo()` del contrato de la víctima; (2) convence al dueño de enviarle un poco de ether a ese contrato, con cualquier excusa; (3) el dueño firma el envío. La cadena queda **dueño → contrato del atacante → contrato de la víctima**, y dentro de la víctima `msg.sender` es el contrato del atacante —la comprobación fallaría—, pero `tx.origin` **sigue siendo el dueño**. La comprobación pasa y el contrato se vacía.

El dueño firmó una cosa y autorizó otra sin enterarse. **Para autorizar siempre se usa `msg.sender`**. La documentación oficial de Solidity lo dice con esas palabras: «nunca use `tx.origin` para autorización». Su único uso legítimo es responder a «¿esta llamada vino directa de una persona o pasó por un contrato?», y hasta eso es frágil.

08 Errores: `require`, `revert`, errores propios y `assert`

En Solidity no hay excepciones que se atrapen y se manejen dentro de la función. Solo hay una cosa: **revertir**. Y revertir significa deshacer todo.

QUÉ SE DESHACE Y QUÉ NO

Se deshace todo. Una transacción de Ethereum es atómica: si se revierte en la última línea, se deshacen todas las escrituras de estado desde el principio, todos los eventos emitidos y todas las transferencias de ether, incluidas las de los contratos que se llamaron en el camino. La cadena queda como si la transacción no hubiera existido: no hay estados intermedios ni «se alcanzó a guardar la mitad».

No se deshace el gas gastado. La transacción *sí* ocurrió: está en el bloque con `Status: Fail`, y quien la firmó pagó todo el trabajo hecho antes de revertir; solo se devuelve el gas que sobró del límite. Por eso una validación cara al final es mala idea: quien se equivoca paga todo lo anterior. **Validar primero, y barato.**

FORMA	QUÉ LLEGA A QUIEN LLAMA	CUÁNDO SE USA
<code>require(cond, "texto")</code>	<code>Error(string)</code>	Lo clásico. Legible, pero la cadena de texto se paga.

<code>revert("texto")</code>	<code>Error(string)</code>	Lo mismo, cuando la condición se lee mejor al revés con un <code>if</code> .
<code>revert MiError(a, b)</code>	<code>MiError(tipo, tipo)</code>	Lo recomendado hoy. Con parámetros. Desde 0.8.4.
<code>assert(cond)</code>	<code>Panic(0x01)</code>	Para invariantes internas, nunca para entradas.

```
error SaldoInsuficiente(uint256 pedido, uint256 disponible);

function conRequire(uint256 pedido) external view {
    require(pedido <= saldo, "saldo insuficiente");
}

function conErrorPropio(uint256 pedido) external view {
    if (pedido > saldo) revert SaldoInsuficiente(pedido, saldo);
}
```

Comparando dos contratos idénticos salvo por eso —uno con `require(v > 100, "el valor debe ser mayor que cien unidades")` y otro con un error propio de dos parámetros—:

MEDICIÓN	<code>require</code> + texto	error propio	DIFERENCIA
Gas de despliegue	105 123	93 223	-11 900 (-11,3 %)
Gas al revertir	3 216	3 177	-39 (-1,2 %)

UN MATIZ QUE CASI NADIE CUENTA BIEN

Se repite mucho que los errores personalizados «ahorran mucho gas al revertir». **Nuestra medición dice que no: al revertir la diferencia son 39 de gas, ruido.** Donde sí se nota es en el **despliegue**: -11 900, porque la cadena de texto viaja dentro del bytecode y se paga al poner el contrato en la cadena. Se paga una vez, pero también cuenta para el límite de 24 576 bytes por contrato (EIP-170), que es un muro real en contratos grandes. Y la razón principal para usarlos no es el gas: es que **llevan parámetros**. No es lo mismo «pago incorrecto» que `PagoIncorrecto(esperado: 1000, recibido: 900)`.

8.1 • Para qué sirve `assert` de verdad

La regla: `require` y `revert` son para lo que el usuario puede hacer mal; `assert` es para lo que *nosotros podemos haber escrito mal*. Un `assert` que falla no dice «el usuario envió un dato inválido», dice «hay un error en este contrato». Nunca debería poder fallar. Solidity lo distingue en el resultado: `require` produce `Error(string)` y `assert` produce `Panic(uint256)` con un código.

CÓDIGO	QUÉ PASÓ
0x01	Falló un <code>assert</code> .
0x11	Desbordamiento aritmético (suma, resta o multiplicación fuera de rango).
0x12	División o módulo por cero.

Cuando una prueba falla con «Panic 0x11», no busquen un `require`: busquen una resta que se fue por debajo de cero, casi siempre `a - b` con `b > a`.

Dato histórico que todavía circula mal: antes de 0.8.0 un `assert` fallido usaba el opcode `INVALID` y **se quedaba con todo el gas** del límite. Desde 0.8.0 usa `REVERT` como todo lo demás y devuelve el sobrante. Si leen que «assert te quema todo el gas», es información vieja.

09 Eventos y logs

Un evento no cambia el estado: escribe una entrada en el **registro** (*log*) de la transacción. Es el único mecanismo que tiene un contrato para contarle algo al mundo de fuera.

```
/// Hasta 3 parámetros `indexed`: son los que se pueden filtrar.
event Transferido(address indexed de, address indexed a, uint256 monto);

emit Transferido(msg.sender, a, monto);
```

EL PUNTO QUE HAY QUE GRABARSE: UN CONTRATO NO PUEDE LEER EVENTOS

Ni los suyos ni los de nadie. Los logs no son accesibles desde la EVM: viven en el recibo de la transacción, fuera del estado. Existen **solo** para quien mira desde fuera —interfaces, indexadores, exploradores, auditorías—.

La regla de decisión: si un dato lo necesita la *lógica* del contrato, va a `storage` y se paga; si lo necesita la *interfaz*, va en un evento y cuesta mucho menos. Emitir un evento creyendo que eso «guarda» el dato es un error de diseño que no da síntomas hasta que otra función lo necesita.

Lo que se ahorra, guardando el mismo `uint256` de las dos maneras:

CÓMO SE GUARDA	GAS	COMENTARIO
Evento con 1 <code>indexed</code>	22 840	Incluye los 21 000 fijos. El log en sí: $375 + 2 \times 375 + 32 \times 8 = 1\,381$.
Escribir un <code>mapping</code> nuevo	43 606	Estrenar ranura: +20 766 sobre el evento.
Actualizar ese <code>mapping</code>	26 506	Ya no estrenar ranura: baja a +3 666.

Cada evento deja hasta cuatro *topics*. El primero es siempre el **hash keccak256 de la firma del evento** (`Transferido(address,address,uint256)`): por eso es idéntico en todos los contratos que declaren ese evento, y por eso un explorador lo reconoce. Los otros tres son los parámetros `indexed`; el resto va concatenado en el campo `data`.

- **Se indexa lo que se va a buscar:** direcciones, identificadores, categorías. «Todas las transferencias hacia esta dirección» es una consulta barata gracias a eso.

- **No se indexa lo que solo se va a leer:** montos, textos. Cada topic cuesta 375 de gas más; los datos no indexados, 8 por byte.
- **Trampa:** si se marca `indexed` un `string` o un `bytes`, en el topic no queda el texto sino su **hash** —un topic son 32 bytes fijos—. Se puede buscar por igualdad exacta, pero el valor original se pierde. Casi nunca es lo que uno quería.
- Un evento `anonymous` no guarda la firma en el topic 0: gana un topic indexado (cuatro) y cuesta un poco menos, pero deja de ser reconocible. Es raro verlo.

10 Modificadores y el `_;`

Un modificador es código que envuelve a una función. El `_;` marca el sitio exacto donde se inserta el cuerpo de la función modificada.

```
modifier soloJefe() {
    if (msg.sender != jefe) revert NoEsJefe(msg.sender);
    _; // aquí se inserta el cuerpo de la función
}

modifier contando() {
    entradas += 1;
    _;
    entradas -= 1; // lo de después del `_;` corre AL SALIR
}

function cambiar(address nuevo) external soloJefe contando { jefe = nuevo; }
```

- Se aplican **en el orden en que se escriben**, de izquierda a derecha, anidándose. Arriba: la comprobación de `soloJefe`, después `entradas += 1`, después el cuerpo, después `entradas -= 1`.
- Si un modificador **no** tiene `_;`, el cuerpo de la función nunca se ejecuta. Compila igual. Es un error clásico y silencioso.
- Pueden recibir argumentos: `modifier libre(uint256 franja)`, usado como `function reservar(uint256 franja) external libre(franja)`.
- Es el mecanismo de `onlyOwner`, `whenNotPaused` y `nonReentrant` de OpenZeppelin.

SU LÍMITE: NO RECIBEN NI DEVUELVEN DATOS DE LA FUNCIÓN

Un modificador no ve los parámetros de la función que envuelve (salvo los que se le pasen explícitamente), no puede leer lo que la función va a devolver, y no puede entregarle un valor calculado. Si la comprobación necesita un dato intermedio —el saldo que ya se leyó, el monto que se acaba de calcular—, el modificador obliga a **leerlo dos veces**, y en Solidity leer dos veces se paga dos veces.

La alternativa: una función `internal` que reciba lo que necesite y devuelva lo que haga falta. Más larga de escribir, más barata de ejecutar y más fácil de seguir. Los modificadores brillan en comprobaciones que no dependen de nada —quién llama, si está pausado, en qué fase está el contrato—; fuera de eso suelen estorbar.

11 Herencia, interfaces, contratos abstractos y bibliotecas

PIEZA	QUÉ ES	SE USA PARA
contract A is B	Herencia. El bytecode desplegado es uno solo , con todo junto.	Reutilizar estado y comportamiento: <code>Ownable</code> , <code>Pausable</code> .
interface	Solo firmas <code>external</code> . Sin estado, sin constructor, sin implementación.	Hablar con un contrato ajeno del que solo se conoce la dirección: <code>IERC20</code> .
abstract contract	Puede tener estado y funciones implementadas, pero al menos una sin cuerpo.	Una base compartida que obliga a los hijos a completar algo. No se despliega sola.
library	Funciones sin estado propio. Si son todas <code>internal</code> , el compilador las incrusta y no hay despliegue aparte.	Cálculos reutilizables: porcentajes, matemáticas, conjuntos.

```
interface IMedible {
    function medir() external view returns (uint256);
}

abstract contract Base {
    /// `virtual`: un hijo puede reemplazarla. Sin cuerpo: el hijo DEBE escribirla.
    function nombre() public view virtual returns (string memory);

    function saludo() public view returns (string memory) {
        return string.concat("hola, ", nombre());
    }
}

library Porcentaje {
    /// Puntos básicos: 100 bps = 1 %. Multiplicar ANTES de dividir.
    function bps(uint256 monto, uint256 puntos) internal pure returns (uint256) {
        return (monto * puntos) / 10_000;
    }
}

contract Hijo is Base, IMedible {
    using Porcentaje for uint256;    // ahora uint256(1000).bps(500) se escribe como método

    function nombre() public pure override returns (string memory) { return "hijo"; }
    function medir() external pure returns (uint256) { return uint256(1000).bps(500); }
}
```

- `virtual` en la madre: «esta se puede reemplazar». `override` en la hija: «esta reemplaza a la de arriba». Los dos son obligatorias: el compilador no deja sobrescribir por accidente. Para llamar a la versión de la madre, `super.f()` .
- `using A for B` pega las funciones de la biblioteca `A` al tipo `B` como si fueran métodos: `monto.bps(500)` es exactamente `Porcentaje.bps(monto, 500)` . Es azúcar sintáctica; no cambia el gas.
- Un `abstract contract` no se puede desplegar, y una `interface` tampoco: se usa para *envolver* una dirección existente, `IERC20(direccion).transfer(...)` .

EL ORDEN DE LINEALIZACIÓN, EN UNA LÍNEA

En `contract C is A, B` las bases se escriben **de la más general a la más derivada** —si el compilador se queja con «Linearization of inheritance graph impossible», casi siempre están al revés— y, en caso de conflicto, **gana la de más a la derecha**: Solidity resuelve la herencia múltiple con C3 recorriendo la lista de derecha a izquierda. En la práctica: `Ownable` y compañía primero, su lógica después. **Si vienen de Python, ojo**: Python también usa C3, pero el orden en que se escriben las bases es **el contrario**; la documentación de Solidity lo advierte expresamente.

12 Ether: recibirlo y enviarlo

`payable` es el interruptor que decide si el dinero puede entrar. Una función sin `payable` que reciba valor hace revertir la transacción sola, sin escribir nada.

FORMA	GAS QUE ENTREGA	SI FALLA	VEREDICTO HOY
<code>a.transfer(monto)</code>	2 300 fijos	Revierte	Desaconsejada . El límite fijo ya no es confiable.
<code>a.send(monto)</code>	2 300 fijos	Devuelve <code>false</code>	Peor aún : si no se comprueba el <code>false</code> , el fallo pasa inadvertido.
<code>a.call{value: m}("")</code>	Todo el disponible	Devuelve <code>false</code>	La recomendada , con una condición: el estado ya debe estar en orden.

```
function retirar() external {
    uint256 monto = pendiente[msg.sender];
    pendiente[msg.sender] = 0;
    (bool ok, ) = payable(msg.sender).call{value: monto}("");
    if (!ok) revert EnvioFallido();
}
```

// 1 · effects
// 2 · interaction

La coma suelta en `(bool ok, )` descarta el segundo valor devuelto —los datos de retorno—, que aquí no interesan.

Durante años el consejo fue el contrario: usar `transfer()`, porque su límite de 2 300 de gas impedía que el receptor ejecutara código y, con eso, impedía la reentrada. Era una defensa que venía puesta.

En 2019 la actualización **Istanbul** incluyó la **EIP-1884**, que subió el precio de varios opcodes —entre ellos `SLOAD`, de 200 a 800—. De un día para otro, contratos correctos que recibían dinero con un `receive()` que leía una sola variable dejaron de caber en 2 300 de gas: los envíos hacia ellos empezaron a revertir y ese dinero quedó inalcanzable. No fue un error de esos contratos; fue que el suelo se movió, y la propia EIP lo anticipa en su sección de compatibilidad. El precio volvió a moverse después: hoy `SLOAD` no cuesta 800 sino **2 100 en frío y 100 en caliente**, por la EIP-2929.

La lección de fondo: **un número mágico de gas escrito en el lenguaje no puede ser una garantía de seguridad**, porque los precios de la EVM cambian —y cambiaron dos veces en cinco años—. Hoy se usa `call`, y la protección contra la reentrada se consigue con el **orden de las operaciones** (checks-effects-interactions) y con el patrón de retiro: el Laboratorio 07.

```
receive() external payable { emit Recibido(msg.sender, msg.value, false); }
fallback() external payable { emit Recibido(msg.sender, msg.value, true); }
```

Ninguna de las dos lleva la palabra `function`, y las dos son `external`. `receive()` corre cuando llega ether **sin datos**; si no existe pero hay un `fallback() payable`, corre ese; si no hay ninguno, revierte. `fallback()` corre cuando llegan datos que **no coinciden con ninguna función**. Este mismo mecanismo —un `fallback` que reenvía toda llamada desconocida a otra dirección con `delegatecall`— es lo que hace posibles los contratos actualizables: potente y peligroso a partes iguales.

UN CONTRATO PUEDE RECIBIR ETHER SIN QUE SU CÓDIGO INTERVENGA

Aunque no declaren `receive` ni `fallback` —y aunque los declaren y hagan `revert`—, hay dos caminos por los que le llega ether a un contrato **sin ejecutar ni una línea suya**: ser el destino de un `selfdestruct` y ser la dirección que recibe las recompensas de un bloque. En los dos casos el saldo sube y el contrato no se entera; la documentación oficial lo dice sin rodeos: no puede reaccionar y por lo tanto tampoco puede rechazarlo.

La consecuencia práctica: **nunca hagan que la lógica dependa de que `address(this).balance` sea exactamente lo que ustedes contabilizaron**. Puede ser mayor. Un contrato que exija `require(address(this).balance == totalRegistrado)` se puede bloquear para siempre desde fuera, gratis, por cualquiera. La contabilidad se lleva en variables propias; el saldo real solo se consulta.

13 Aritmética: sin desbordes silenciosos y sin decimales

Desde Solidity 0.8.0, toda operación aritmética que se desborde revierte con `Panic(0x11)`. Antes daba la vuelta en silencio: en un `uint8`, `255 + 1` era `0` y `0 - 1` era `255`. De ahí salieron robos famosos, y por eso existía `SafeMath`, que hoy no hace falta y no debería aparecer en código nuevo.

```
function desborda(uint8 x) external pure returns (uint8) {
    return x + 1; // con x = 255 REVIERTE (Panic 0x11)
}

function sinRevisar(uint8 x) external pure returns (uint8) {
    unchecked { return x + 1; } // con x = 255 devuelve 0, en silencio
}
```

CUÁNDO SE JUSTIFICA `unchecked`

`unchecked` apaga la comprobación dentro de sus llaves y recupera el comportamiento viejo. Cada comprobación cuesta unas pocas decenas de gas, así que la tentación de apagarla «para optimizar» es grande y casi siempre mala. Solo se justifica cuando ustedes pueden **demostrar** que el desbordamiento es imposible, y conviene escribir esa demostración en un comentario al lado. El caso de libro es el contador de un bucle acotado:

```
for (uint256 i = 0; i < xs.length; ) {
    s += xs[i];
    unchecked { ++i; } // i no puede desbordar: el límite es xs.length
}
```

`i` nunca pasa de `xs.length`: desbordar un `uint256` es físicamente imposible. Fuera de casos así de claros —y de restas donde un `require` anterior ya garantizó el orden—, dejen la comprobación puesta. Lo que se ahorra son decenas de gas; lo que se arriesga es todo.

13.1 • No hay decimales. Ninguno.

Solidity declara tipos de punto fijo (`fixed`, `ufixed`) pero **no están implementados**: se pueden declarar y no se pueden usar. De punto flotante no hay ni la promesa. `1 / 3` es `0`. Una máquina que debe dar el mismo resultado en miles de nodos no se puede permitir el redondeo de los flotantes.

La respuesta del ecosistema es trabajar siempre con **la unidad más pequeña, en enteros**: `1 wei = 1`; `1 gwei = 109 wei` (para hablar de precios de gas); `1 ether = 1018 wei` (para hablar con humanos, y **solo se convierte en la interfaz**). Un ERC-20 hace lo mismo con su campo `decimals`: si vale 18, «1,5 fichas» se guarda como `1500000000000000000`. La cadena solo ve enteros; la coma la pone la interfaz.

MULTIPLICAR ANTES DE DIVIDIR

Como la división entera descarta el resto, el orden cambia el resultado. El 3 % de 250:

```
return (monto / 100) * 3;    // 250/100 = 2 -> 6. Se perdió el resto.
return (monto * 3) / 100;    // 250*3/100 = 7. Multiplicar ANTES de dividir.
```

6 frente a 7. En wei, esa diferencia repetida en miles de operaciones es dinero real que se evapora.

Multiplicar siempre primero, vigilando lo único que eso puede provocar: que el producto intermedio se desborde. Con `uint256` hay muchísimo margen, pero si trabajan en `uint64` o `uint128`, conviertan a `uint256` antes de multiplicar.

La convención del sector para los porcentajes son los **puntos básicos** (*bps*): 10 000 bps = 100 %, así que 1 % son 100 bps y 0,25 % son 25 bps. Permite expresar fracciones finas sin decimales: `(monto * bps) / 10_000`. El guion bajo de `10_000` es solo para leerlo mejor; el compilador lo ignora.

14 Los errores del primer mes

SÍNTOMA → CAUSA → ARREGLO

SÍNTOMA	CAUSA	ARREGLO
La función corre, no falla, y el estado no cambió	Declararon <code>Struct memory x = ...</code> en lugar de <code>storage</code> : modificaron una copia.	Cambiar <code>memory</code> por <code>storage</code> . Si no hay que modificar nada, <code>storage</code> también sirve y evita la copia.
Operator <code>==</code> not compatible with types <code>string memory</code>	Compararon dos <code>string</code> con <code>==</code> .	<code>keccak256(bytes(a)) == keccak256(bytes(b))</code> . Para «¿está vacío?», <code>bytes(a).length == 0</code> .
«Ya registré ese dato» y el contrato dice que no existe	Un <code>mapping</code> devuelve cero para toda clave: no distingue «vacío» de «nunca puesto».	Guardar un <code>bool existe</code> en el <code>struct</code> y comprobarlo.
Quieren recorrer un <code>mapping</code> y no encuentran cómo	No se puede: no guarda las claves.	Mantener un <code>address[]</code> aparte con <code>push</code> a mano. Cuesta gas: decidan si de verdad hace falta.
Panic 0x11 en una resta que «no puede fallar»	Resta por debajo de cero en un <code>uint</code> : <code>a - b</code> con <code>b > a</code> . No hay negativos.	Comprobar el orden antes (<code>if (b > a) revert ...</code>) o replantear el cálculo.
Un porcentaje da cero o un número raro	Dividieron antes de multiplicar; no hay decimales.	<code>(monto * puntos) / 10_000</code> . Nunca <code>(monto / 10_000) * puntos</code> .
Una fecha o un contador dan un número absurdo	Truncamiento silencioso al convertir a un tipo más pequeño (<code>uint32(block.timestamp)</code>).	Usar un tipo que alcance (<code>uint64</code> para marcas de tiempo) y convertir solo cuando se sepa que cabe.
La función tiene un modificador y el cuerpo nunca se ejecuta	Al modificador le falta el <code>;</code> .	Poner el <code>;</code> donde debe insertarse el cuerpo. Sin él compila igual: hay que mirarlo.

La interfaz dice que guardó el dato y el contrato no lo tiene	Se emitió un evento creyendo que eso lo guarda. Un contrato no puede leer eventos.	Si lo necesita la lógica, va a <code>storage</code> . El evento es solo para quien mira desde fuera.
El envío de ether falla siempre contra cierto contrato	Usaron <code>transfer()</code> / <code>send()</code> y al receptor no le alcanzan los 2 300 de gas (EIP-1884).	Usar <code>call{value: m}("")</code> con el estado ya actualizado antes, y comprobar el <code>bool</code> .
Guardaron algo «privado» y apareció publicado	<code>private</code> limita quién llama, no quién ve. El storage se lee con <code>eth_getStorageAt</code> .	No guardar secretos en cadena. Guardar el hash y revelar después.
La verificación en el explorador falla «sin haber cambiado nada»	Compilador, optimización o <i>runs</i> distintos a los del despliegue. Con <code>^0.8.0</code> ni se sabe cuál se usó.	Fijar <code>pragma solidity 0.8.34;</code> y anotar compilador, optimización y runs al desplegar.
No arguments passed to the base constructor al heredar de <code>Ownable</code>	OpenZeppelin 5 exige el dueño inicial; en la versión 4 ese parámetro no existía.	<code>constructor(...)</code> <code>Ownable(msg.sender) { ... }</code> . Casi todos los tutoriales de internet son de la 4.
Member "increaseAllowance" not found en un ERC-20	OpenZeppelin 5 eliminó <code>increaseAllowance</code> y <code>decreaseAllowance</code> de ERC20: no eran del estándar. Nada las reemplaza.	<code>approve</code> con el valor exacto nuevo. Para operar sobre tokens <i>ajenos</i> existen <code>safeIncreaseAllowance</code> y <code>safeDecreaseAllowance</code> en <code>SafeERC20</code> (comprobado en la 5.6.1 del curso).
<code>DocstringParsingError: Documentation tag @... not valid</code>	Una línea del bloque NatSpec <i>empieza</i> con una arroba que no es etiqueta conocida (un correo partido, una mención).	Que la arroba no quede al principio de la línea, o usar una etiqueta válida (<code>@custom:algo</code>). A mitad de línea no molesta.

15 Un contrato de referencia, comentado

COMPILA CON 0.8.34

Reserva de la sala de cómputo: cada franja horaria se reserva una sola vez y se paga por adelantado; cancelar devuelve la mitad. Es pequeño a propósito, y tiene una pieza de casi cada sección de esta guía.

Lectura complementaria: en el repositorio está `contracts/demo/Tablon.sol`, el tablón de anuncios que se alquila por tiempo que se explica en clase. Es otro contrato distinto, comentado línea por línea y con 21 pruebas en verde. Este de aquí es más corto y sirve de plantilla; ese otro sirve para leer con calma la *por qué* de cada decisión.

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

/// @title Reserva de la sala de cómputo
/// @notice Cada franja se reserva una vez, se paga por adelantado y cancelar devuelve la mitad.
/// @dev CONTRATO DE REFERENCIA DE LA GUÍA. Sin auditar, nunca con dinero real.
contract ReservaSala is Ownable {
    enum Estado { Libre, Reservada, Cancelada } // 0, 1, 2

    struct Reserva {
        address quien; // 20 bytes
        uint64 hechaEn; // 8 bytes } 31 de 32: UNA sola ranura
        uint16 personas; // 2 bytes
        Estado estado; // 1 byte
    }

    uint16 public constant CUPO = 30; // se sabe al escribir
    uint256 public immutable precioFranja; // se decide al desplegar

    mapping(uint256 franja => Reserva datos) public reservas;
    mapping(address quien => uint256 wei) public saldo;
    uint256[] private _franjasUsadas; // el mapping no se recorre

    event Reservada(uint256 indexed franja, address indexed quien, uint16 personas);
    event Cancelada(uint256 indexed franja, address indexed quien, uint256 reembolso);
    event Retirado(address indexed quien, uint256 monto);

    error FranjaOcupada(uint256 franja, address porQuien);
    error PersonasFueraDeRango(uint16 recibido, uint16 cupo);
    error PagoIncorrecto(uint256 esperado, uint256 recibido);
    error NoEsSuya(uint256 franja);
    error SinSaldo();
    error EnvioFallido();
    error NoRecibeEnviosSueitos();

    constructor(uint256 precioFranja_) Ownable(msg.sender) {
        precioFranja = precioFranja_; // única escritura posible
    }

    modifier libre(uint256 franja) {
        Reserva storage r = reservas[franja];
        if (r.estado == Estado.Reservada) revert FranjaOcupada(franja, r.quien);
        _; // aquí entra la función
    }

    /// @notice Reserva una franja pagando exactamente `precioFranja` wei.
    function reservar(uint256 franja, uint16 personas) external payable libre(franja) {
        if (personas == 0 || personas > CUPO) revert PersonasFueraDeRango(personas, CUPO);
        if (msg.value != precioFranja) revert PagoIncorrecto(precioFranja, msg.value);

        if (reservas[franja].hechaEn == 0) _franjasUsadas.push(franja);
        reservas[franja] = Reserva({
            quien: msg.sender, hechaEn: uint64(block.timestamp),
            personas: personas, estado: Estado.Reservada
        });
        saldo[owner()] += msg.value; // se acredita, no se envía
        emit Reservada(franja, msg.sender, personas);
    }

    /// @notice Cancela su propia reserva. Devuelve la mitad de lo pagado.
    function cancelar(uint256 franja) external {
        Reserva storage r = reservas[franja]; // `storage`: queda escrito
        if (r.quien != msg.sender || r.estado != Estado.Reservada) revert NoEsSuya(franja);
    }
}

```

```

        r.estado = Estado.Cancelada;

        uint256 devolver = precioFranja / 2;           // enteros: 3 wei / 2 = 1
        uint256 caja = saldo[owner()];
        if (devolver > caja) devolver = caja;
        saldo[owner()] = caja - devolver;
        saldo[msg.sender] += devolver;
        emit Cancelada(franja, msg.sender, devolver);
    }

    /// @notice Retira el saldo propio. Checks → Effects → Interactions.
    function retirar() external {
        uint256 monto = saldo[msg.sender];
        if (monto == 0) revert SinSaldo();

        saldo[msg.sender] = 0;                          // EFFECTS
        (bool ok, ) = payable(msg.sender).call{value: monto}(""); // INTERACTION
        if (!ok) revert EnvioFallido();
        emit Retirado(msg.sender, monto);
    }

    function franjasUsadas() external view returns (uint256) { return _franjasUsadas.length; }

    /// @notice Ocupación en porcentaje entero. Multiplicar ANTES de dividir.
    function ocupacion(uint256 franjasDelDia) external view returns (uint256) {
        if (franjasDelDia == 0) return 0;                // nunca dividir por cero
        return (_franjasUsadas.length * 100) / franjasDelDia;
    }

    /// @dev Aun rechazando, el contrato puede recibir ether a la fuerza: nunca
    ///        confíen en `address(this).balance` como contabilidad.
    receive() external payable { revert NoRecibeEnviosSueitos(); }
    fallback() external payable { revert NoRecibeEnviosSueitos(); }
}

```

CINCO PREGUNTAS SOBRE ESTE CONTRATO

Se responden leyéndolo, sin desplegarlo. Buen repaso antes de un parcial.

1. ¿Cuántas ranuras de storage escribe `reservar` la primera vez que se usa una franja? ¿Y la segunda vez que se reserva *esa misma* franja después de cancelarla?
2. `CUP0` es `constant` y `precioFranja` es `immutable`. ¿Por qué no puede ser al revés?
3. En `cancelar` se usa `Reserva storage r`. ¿Qué pasaría exactamente con `memory`, y lo notaría alguna prueba evidente?
4. `_franjasUsadas` crece y nunca se reduce. ¿Qué pasa si alguien reserva 100 000 franjas? ¿Alguna función deja de servir?
5. El `modifier libre` lee `reservas[franja]` y `reservar` la vuelve a leer en la línea del `if`. ¿Cuánto cuesta esa segunda lectura y cómo se evitaría? (Sección 10.)

16 Glosario

TÉRMINO

QUÉ SIGNIFICA

ABI	El «contrato de comunicación»: qué funciones tiene y con qué tipos. Lo que necesita una interfaz para hablarle.
Bytecode	El resultado de compilar: lo que de verdad queda en la cadena. Por eso se verifica el código fuente.
EIP / ERC	Propuesta numerada de cambio a Ethereum. Un ERC es una EIP de estándares de aplicación (ERC-20, ERC-721).
EOA	Cuenta controlada por una llave privada: una persona, no un contrato.
EVM	La máquina que ejecuta el bytecode, igual en todos los nodos.
Gas	Unidad de trabajo. Cada opcode cuesta una cantidad fija; el precio en ether lo pone el mercado.
keccak256	La función hash de Ethereum. Variante de SHA-3, no idéntica al SHA3-256 estándar.
Log / evento	Entrada en el recibo de la transacción. La lee quien mira desde fuera; el contrato no.
NatSpec	Comentarios <code>@notice</code> , <code>@param</code> , <code>@dev</code> . Van al ABI y los muestran las billeteras.
Opcode	Instrucción de la EVM: <code>SLOAD</code> , <code>SSTORE</code> , <code>CALL</code> ...
Panic	Error del propio lenguaje, con código: desbordamiento, división por cero, <code>assert</code> fallido.
Ranura (slot)	Cada casilla de 32 bytes del storage, numeradas desde 0.
Revertir	Abortar deshaciendo todos los cambios de la transacción. No devuelve el gas ya gastado.
Selector	Los 4 primeros bytes del hash de la firma de una función. Identifica a cuál se llama.
SPDX	Lista estándar de identificadores de licencia. Va en la primera línea del archivo.
Topic	Cada campo indexado de un log. Hasta cuatro; el primero es la firma del evento.
wei	La unidad mínima de ether. 1 ether = 10 ¹⁸ wei. La cadena solo trabaja en wei.

17 Documentación oficial

ENLACES VERIFICADOS

Cuando esta guía y la documentación oficial no coincidan, **manda la documentación oficial**: avísenle al docente y se corrige. Todos estos enlaces se abrieron y funcionan sin cuenta.

Solidity 0.8.34 — todas las páginas cuelgan de <https://docs.soliditylang.org/en/v0.8.34/>:

- `types.html` — tipos, conversiones, valores por defecto. • `cheatsheet.html` — resumen de una página.
- `contracts.html` — visibilidad, herencia y linealización, eventos, `receive` / `fallback`.
- `control-structures.html` — errores, `require`, `assert` y los códigos de `Panic`.
- `units-and-global-variables.html` — `msg`, `block`, `tx`, `wei`, `ether`.
- `security-considerations.html` — por qué `private` no es secreto y `tx.origin` no autoriza.
- `internals/layout_in_storage.html` — ranuras y empaquetado del storage.

- `natspec-format.html` · `080-breaking-changes.html` · `common-patterns.html` (retiro, control de acceso, máquina de estados).

OpenZeppelin Contracts 5.x — <https://docs.openzeppelin.com/contracts/5.x/>, con `Ownable` en `/api/access` y el token en `/api/token/erc20`. Lo que se eliminó al pasar de la 4 a la 5 está en las notas de versión: <https://github.com/OpenZeppelin/openzeppelin-contracts/releases/tag/v5.0.0>

Las EIP que se citan aquí — <https://eips.ethereum.org/EIPS/eip-NNNN>:

- **1884** (2019) · subió `SLOAD` de 200 a 800 y dejó sin piso a `transfer()`.
- **2929** (2021) · los precios vigentes: 2 100 en frío, 100 en caliente, 2 900 para modificar una ranura.
- **2200** · reglas de medición de `SSTORE`. · **2028** · bajó el byte no cero de `calldata` de 68 a 16.
- **170** · límite de 24 576 bytes de bytecode por contrato.

Y dos referencias de consulta rápida: la lista oficial de licencias, <https://spdx.org/licenses/>, y el precio de cada opcode de la EVM, <https://www.evm.codes/>

CÓMO SE MIDIERON LAS CIFRAS DE ESTA GUÍA

Todas las cifras de gas salen de contratos pequeños desplegados en la red local de Hardhat 3.16.0, con Solidity 0.8.34 y el optimizador en 200 ejecuciones —la configuración congelada del curso, en `versiones.md`—, leyendo el `gasUsed` del recibo de cada transacción. Las lecturas de ranuras se hicieron con `eth_getStorageAt`. Ustedes pueden repetir cualquiera de estas mediciones: la red local es determinista y da los mismos números.

Las cifras se midieron primero con `0.8.28` (objetivo `cancun`) y, al cambiar el curso de compilador, se **volvieron a medir todas** con `0.8.34` (objetivo `osaka`) compilando los mismos contratos: **ninguna cambió**. Que el compilador cambie no obliga a que el gas cambie —lo que cambia el precio del gas son las bifurcaciones de la red, no el compilador—, pero eso *se comprueba*, no se supone: es exactamente el hábito que esta guía les pide.